UNIVERSITÀ DELLA CALABRIA

Anno accademico 2026/2027

Algorithmic Game Theory

Appunti del corso: teoria e laboratori

Corso tenuto da Sebastiano A. Piccolo

Ultima compilazione: 3 ottobre 2026

Indice

1 Introduzione all'Algorithmic Game Theory **1**

1.1 Informazioni sul corso . . . 1

1.2 Che cos'è la teoria dei giochi . . . 2

1.3 Che cos'è l'Algorithmic Game Theory . . . 2

1.4 Panoramica delle aree dell'AGT . . . 2

1.4.1 Giochi non cooperativi . . . 2

1.4.2 Giochi coalizionali . . . 4

1.4.3 Scelta sociale computazionale . . . 5

1.4.4 Mechanism design . . . 5

1.4.5 Matching markets . . . 6

1.5 Giochi combinatori . . . 6

1.5.1 Dalla teoria dei giochi al giocare . . . 6

1.5.2 L'albero di gioco . . . 6

1.5.3 Un'IA per il tris . . . 7

1.5.4 L'algoritmo Minimax . . . 7

1.5.5 I limiti di Minimax . . . 8

1.5.6 Potatura Alpha-Beta . . . 8

1.5.7 Oltre alpha-beta . . . 10

1.6 Laboratorio: Minimax per il tris . . . 11

1.6.1 Rappresentazione e condizioni di vittoria . . . 11

1.6.2 Minimax . . . 11

1.6.3 Scegliere la mossa migliore . . . 12

1.6.4 Minimax contro sé stesso . . . 13

1.6.5 Minimax con potatura alpha-beta . . . 13

1.6.6 Giocare contro l'IA . . . 14

2 Teoria dell'utilità **15**

2.1 Concetti di base . . . 15

2.1.1 Alternative . . . 15

2.1.2 Preferenze . . . 16

2.2 Rappresentare le preferenze: utilità ordinale 16
2.2.1 Le condizioni sono necessarie 16
2.2.2 Le condizioni sono sufficienti 17
2.2.3 Esempi: l'utilità ordinale non è interpretabile 17
2.3 Oltre l'ordinale: le lotterie 18
2.3.1 Lotterie composte 18
2.3.2 Valore atteso 18
2.3.3 Utilità attesa 19
2.4 L'utilità di Von Neumann–Morgenstern 19
2.4.1 Necessità degli assiomi 20
2.4.2 Sufficienza: dimostrazione del teorema VNM 20
2.5 Il paradosso di Allais 21
2.6 Atteggiamento verso il rischio 22
2.7 Applicazioni 23
2.7.1 Condivisione del rischio (*risk sharing*) 24
2.7.2 Assicurazione 24
2.8 Riepilogo sulla teoria VNM 25
2.9 Laboratorio: Battaglia navale e ricerca bayesiana 26
2.9.1 Regole e impostazione 26
2.9.2 Strategia 1: tiro casuale 27
2.9.3 Strategia 2: *hunt and target* 27
2.9.4 Strategia 3: ricerca bayesiana 28
2.9.5 Risultati misurati 30
2.9.6 Che cosa portarsi dietro 31

LEZIONE 1

Introduzione all'Algorithmic Game Theory

TEORIA

1.1 Informazioni sul corso

- **Lezioni**: mercoledì 10:30–13:30 (teoria), giovedì 11:30–13:30 (laboratorio).
- **Teams**: *Algorithmic Game Theory 2026/27* (il codice per entrare è nelle slide).
- **Esame**: un **progetto** in cui si deve *formalizzare* un problema e *implementarne* una soluzione usando gli strumenti dell'AGT.

> **Osservazione (conseguenza pratica).** Dato che l'esame è un progetto, contano due cose: saper *modellare* un problema (chi sono i giocatori, quali azioni hanno, quali utilità) e saper *scrivere codice* che calcola il concetto di soluzione giusto. I laboratori vanno quindi studiati quanto la teoria.

Programma (provvisorio)

1. Giochi combinatori (*Combinatorial Games*)
2. Teoria dell'utilità (*Utility theory*)
3. Teoria dei giochi non cooperativi (*Non-cooperative game theory*)
4. Teoria dei giochi coalizionali (*Coalitional game theory*)
5. Scelta sociale computazionale (*Computational Social Choice*)
6. Mercati di abbinamento (*Matching Markets*)
7. Progettazione di meccanismi (*Mechanism Design*)
8. Divisione equa (*Fair division*)

Testi di riferimento

Non esiste un unico libro che copra tutto il corso; queste sono le risorse gratuite indicate:

- Y. Shoham, K. Leyton-Brown, *Multiagent Systems*;
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani, *Algorithmic Game Theory*;
- A. Karlin, Y. Peres, *Game Theory, Alive*;
- F. Brandt et al., *Handbook of Computational Social Choice*.

1.2 Che cos'è la teoria dei giochi

> **Definizione 1.1 (Teoria dei giochi).** La **teoria dei giochi** studia le *decisioni prese da più soggetti* (*multiperson decision making*), con lo scopo di prevedere come più agenti **razionali** interagiranno e si comporteranno in situazioni di **conflitto** e/o di **cooperazione**.

I quattro ingredienti della definizione:

1. **Conflitto**: le azioni di un giocatore influenzano gli altri.
2. **Cooperazione**: i giocatori possono unire le forze per raggiungere un obiettivo comune.
3. **Comportamento razionale**: ogni giocatore vuole *massimizzare* la propria **utilità attesa** (*expected utility*).
4. **Predizione**: vogliamo sapere cosa succederà. Lo si fa tramite i **concetti di soluzione** (*solution concepts*), come l'equilibrio di Nash o il valore di Shapley.

1.3 Che cos'è l'Algorithmic Game Theory

> **Definizione 1.2 (Algorithmic Game Theory).** L'**Algorithmic Game Theory** (AGT) è l'area all'intersezione tra *teoria dei giochi* e *progettazione di algoritmi*, il cui obiettivo è progettare algoritmi in *ambienti strategici*. [Nisan et al. 2007]

In concreto, l'AGT si occupa di progettare algoritmi per:

- **calcolare** in modo efficiente i concetti di soluzione (es. trovare un equilibrio);
- **aggregare** voti e preferenze;
- **allocare** risorse scarse.

1.4 Panoramica delle aree dell'AGT

1.4.1 Giochi non cooperativi

I giochi non cooperativi modellano situazioni di **conflitto**: i giocatori competono tra loro e non possono necessariamente cooperare. Esempi: scacchi, sasso-carta-forbici, dilemma del prigioniero.

L'obiettivo è modellare una situazione e capire cosa succede. Il concetto di soluzione tipico è l'**equilibrio**.

Il dilemma del prigioniero

Due giocatori scelgono simultaneamente se *cooperare* (C) o *tradire* (D, *defect*). I numeri sono le utilità (anni di carcere con segno negativo): in ogni cella il primo valore è di P1, il secondo di P2.

		P2	
		Cooperate	Defect
P1	Cooperate	$-1, -1$	$-5, 0$
	Defect	$0, -5$	$-3, -3$

Tabella 1.1: Dilemma del prigioniero in *forma normale* (gioco bi-matrice).

Questa rappresentazione a tabella si chiama **forma normale** (o gioco *bi-matrice*): una matrice di utilità per ciascuno dei due giocatori.

Definizione 1.3 (Ottimo di Pareto). Un esito è **Pareto-ottimo** se non esiste un altro esito in cui *qualcuno* sta meglio e *nessuno* sta peggio. In altre parole: non si può migliorare la situazione di un giocatore senza peggiorare quella di qualcun altro.

Definizione 1.4 (Equilibrio di Nash, idea). Un profilo di strategie è un **equilibrio di Nash** se nessun giocatore può migliorare la propria utilità cambiando *da solo* la propria scelta.

Esempio (analisi del dilemma). Nash. Per P1, tradire conviene *qualunque* cosa faccia P2: se P2 coopera, $0 > -1$; se P2 tradisce, $-3 > -5$. Lo stesso vale per P2. Quindi (D, D) è l'unico equilibrio di Nash, con utilità $(-3, -3)$. Da (C, C), invece, ciascuno ha interesse a deviare (passa da -1 a 0). **Pareto.** (C, C) con $(-1, -1)$ è Pareto-ottimo. L'equilibrio (D, D) *non* lo è: (C, C) fa stare meglio entrambi.

Il dilemma mostra la tensione centrale della teoria dei giochi:

$$\textbf{Ottimalità di Pareto} \Rightarrow \textbf{efficienza} \qquad \textbf{Equilibrio di Nash} \Rightarrow \textbf{stabilità}$$

L'esito efficiente non è stabile e l'esito stabile non è efficiente.

Attenzione (definizione nelle slide). Le slide definiscono l'ottimo di Pareto come *"nessun giocatore può deviare senza peggiorare qualcun altro"*. È una formulazione informale: l'ottimo di Pareto *non* riguarda le deviazioni unilaterali (quelle sono il criterio di Nash), ma il confronto tra *esiti*. Nel dilemma sono Pareto-ottimi anche (C, D) e (D, C): passando da $(-5, 0)$ a qualunque altro esito P2 starebbe peggio. L'unico esito *non* Pareto-ottimo è (D, D).

Applicazioni

- algoritmi per giocare: scacchi, poker, . . . ;
- *Generative Adversarial Networks* (GAN): generatore e discriminatore sono due giocatori;
- sicurezza: pattugliamento, sorveglianza, . . . ;
- decisioni sequenziali robuste (*robust sequential decision making*).

1.4.2 Giochi coalizionali

Nei giochi coalizionali (o cooperativi) i giocatori **uniscono le forze** per un obiettivo comune, ad esempio mettono insieme i soldi per comprare un bene. Le domande sono due:

- quale coalizione è più probabile che si formi?
- come si divide il bene (il *payoff*) tra i membri?

Concetto di soluzione principale: il **valore di Shapley**.

Esempio (tre imprese). Tre imprese A, B, C possiedono ciascuna una parte della tecnologia per sviluppare un prodotto. Due imprese insieme guadagnano 100; una da sola guadagna 0; tutte e tre insieme guadagnano comunque 100. La **funzione caratteristica** v assegna a ogni coalizione il suo valore:

$$\begin{aligned} v(\{A\}) = v(\{B\}) = v(\{C\}) &= 0, \\ v(\{A,B\}) = v(\{A,C\}) = v(\{B,C\}) &= 100, \\ v(\{A,B,C\}) &= 100. \end{aligned}$$

- Se si forma la coalizione di tutti e tre e si divide in parti uguali: 33,33 a testa.
- Ma B e C da soli ottengono 100 e, dividendo, 50 a testa: stanno meglio abbandonando A. La divisione in parti uguali della grande coalizione non è stabile.

Esempio (variante: un giocatore più importante). Ora $v(\{A,B\}) = v(\{A,C\}) = 100$ ma $v(\{B,C\}) = 0$ (il resto invariato). Senza A non si produce nulla: A è più "importante" e la divisione in parti uguali non è più equa. Come dividere in modo equo? Con il **valore di Shapley**.

Approfondimento (calcolo del valore di Shapley nella variante). Il valore di Shapley (sarà definito formalmente più avanti) dà a ogni giocatore il suo *contributo marginale medio* su tutti gli ordini in cui i giocatori possono entrare nella coalizione. Con 3 giocatori gli ordini sono $3! = 6$:

Ordine	contributo di A	contributo di B	contributo di C
A, B, C	0	100	0
A, C, B	0	0	100
B, A, C	100	0	0
B, C, A	100	0	0
C, A, B	100	0	0
C, B, A	100	0	0
Media	66,67	16,67	16,67

Esempio di lettura: nell'ordine B, C, A, A entra quando ci sono già B e C, e porta il valore da $v(\{B,C\}) = 0$ a $v(\{A,B,C\}) = 100$: contributo 100. Il totale è 100, cioè tutto il valore della grande coalizione. Nel caso simmetrico di prima si ottiene invece 33,33 a testa.

Applicazioni

- divisione equa (*fair division*);
- classifiche di importanza, potere di voto, . . . ;
- spiegabilità dell'IA: **SHAP** (valori di Shapley applicati alle feature di un modello);
- *network science*: massimizzazione dell'influenza, centralità, . . .

1.4.3 Scelta sociale computazionale

La **scelta sociale computazionale** (*Computational Social Choice*) si occupa del calcolo dei risultati delle regole di voto e dei problemi legati a rappresentare e raccogliere le preferenze dei giocatori. Domande tipiche:

- come si decide il vincitore di un'elezione?
- a quali condizioni la democrazia è possibile?

Esempio (paradosso di Condorcet). Tre votanti ordinano tre candidati:

Votante	1ª preferenza	2ª preferenza	3ª preferenza
Votante 1	A	B	C
Votante 2	B	C	A
Votante 3	C	A	B

Chi dovrebbe vincere? Confrontiamo i candidati a coppie, a maggioranza:

- A batte B (votanti 1 e 3);
- B batte C (votanti 1 e 2);
- C batte A (votanti 2 e 3).

La preferenza collettiva è **ciclica**: $A \succ B \succ C \succ A$. Anche se ogni votante ha preferenze coerenti (transitive), la maggioranza non le ha, e nessun candidato batte tutti gli altri.

Applicazioni

Elezioni, decisioni collettive, e nel machine learning i *random forest* e gli *ensemble* di modelli (ogni modello "vota" una predizione).

1.4.4 Mechanism design

Il **mechanism design** è una sorta di teoria dei giochi *al contrario*: invece di prendere un gioco come dato e analizzarne gli esiti, si vuole *progettare* un gioco che incentivi i giocatori a comportarsi nel modo desiderato.

Esempio (aste). In un'asta si vuole che le persone dichiarino il *vero* prezzo a cui comprerebbero un oggetto o venderebbero un servizio.

Casi di successo: eBay, Google Ads, appalti pubblici e aste.

1.4.5 Matching markets

Nei **mercati di abbinamento** si vogliono abbinare agenti ad altri agenti, o agenti a oggetti, beni, servizi, garantendo qualche proprietà sociale desiderabile. Esempi: studenti e alloggi universitari, pazienti e donatori di rene, lavoratori e compiti. Nel corso si studierà un algoritmo premiato con il Nobel per lo **stable matching**.

1.5 Giochi combinatori

Definizione 1.5 (Gioco combinatorio). Un **gioco combinatorio** ha le seguenti proprietà:

1. **due giocatori**, di solito chiamati Giocatore 1 e Giocatore 2, oppure MAX e MIN;
2. **informazione perfetta**: entrambi conoscono l'intero stato del gioco;
3. **nessun elemento casuale**: niente dadi, mazzi mescolati o eventi aleatori;
4. **finito**: il gioco termina sempre (niente partite infinite);
5. **turni alternati**: i giocatori muovono uno alla volta;
6. **somma zero**: il guadagno di un giocatore è esattamente la perdita dell'altro.

Esempi: scacchi, dama, go, tris (*tic-tac-toe*).

1.5.1 Dalla teoria dei giochi al giocare

Problema centrale: dato lo stato attuale del gioco, qual è la mossa *migliore*?

La mossa migliore è quella che porta al miglior esito possibile *assumendo che anche l'avversario giochi in modo perfetto*. L'idea chiave è modellare l'avversario come intelligente e capace di dare sempre la contromossa migliore.

Osservazione (perché assumere un avversario perfetto). Se un giocatore vince contro la **risposta migliore** (*best response*) dell'avversario, vince anche contro qualunque risposta non ottimale. Ragionare sul caso peggiore dà quindi una *garanzia*: il risultato non può essere peggiore di quello calcolato.

1.5.2 L'albero di gioco

Definizione 1.6 (Albero di gioco). Ogni gioco di questo tipo si può rappresentare come un **albero di gioco**:

- i **nodi** sono stati del gioco (una configurazione della scacchiera);
- gli **archi** sono mosse, che portano da uno stato a un altro;
- la **radice** è lo stato attuale;

- le **foglie** sono stati terminali (vittoria di qualcuno o pareggio).

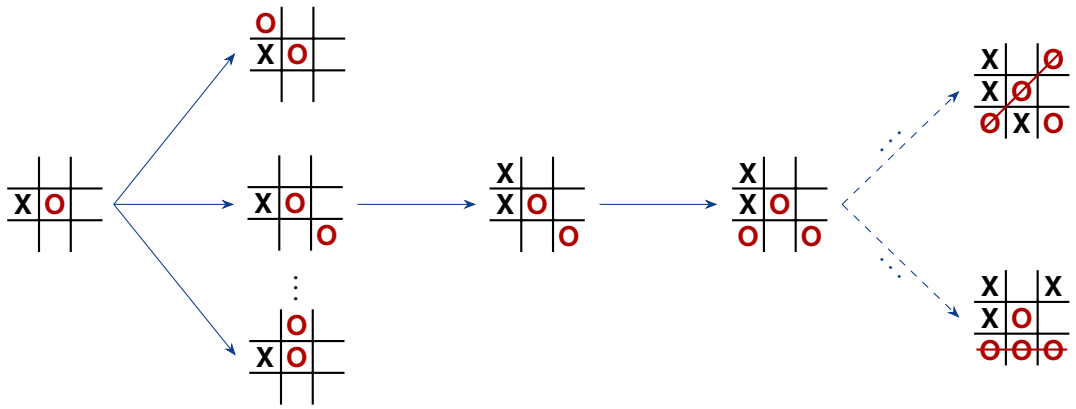


Figura 1.1: Una porzione dell'albero di gioco del tris. A destra due foglie in cui vince O.

1.5.3 Un'IA per il tris

Servono tre elementi:

1. un modo per **rappresentare** la scacchiera e lo stato del gioco;
2. una **funzione di valutazione** per sapere chi vince;
3. una **strategia di ricerca** nell'albero di gioco.

Funzione di punteggio. Il gioco è piccolo e a somma zero, quindi si assegna un punteggio agli stati finali:

$$\text{valore}(s) = \begin{cases} +1 & \text{se vince MAX,} \\ -1 & \text{se vince MIN,} \\ 0 & \text{in caso di pareggio.} \end{cases}$$

La grandezza dei numeri non conta: conta solo l'*ordine* ($+1 > 0 > -1$).

Rappresentazione. La scacchiera è una matrice 3×3 di numeri: X $\to 1$, O $\to 2$, cella vuota $\to 0$. Servono una funzione che controlla se c'è un tris su una riga, colonna o diagonale, e una che controlla se la scacchiera è piena.

1.5.4 L'algoritmo Minimax

Minimax è un algoritmo ricorsivo che esplora l'albero di gioco per determinare la mossa ottimale.

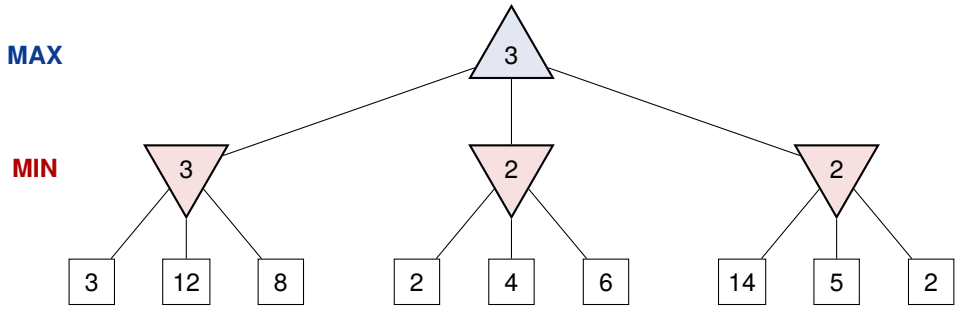
- Il giocatore **MAX** vuole *massimizzare* il punteggio: sceglie la mossa con il valore più alto restituito dalla chiamata ricorsiva.
- Il giocatore **MIN** vuole *minimizzare* il punteggio (che è quello di MAX): sceglie la mossa con il valore più basso.

```
def minimax(board, player):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN)
            best = max(val, best)
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX)
            best = min(val, best)
        return best
```

In formula, il valore minimax di uno stato s è:

$$\mathrm{MM}(s) = \begin{cases} \text{valore}(s) & \text{se } s \text{ è terminale,} \\ \max_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MAX,} \\ \min_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MIN.} \end{cases}$$

Approfondimento (esempio numerico). Un albero a due livelli (MAX alla radice, MIN al livello sotto). Si parte dalle foglie e si risale: ogni nodo MIN prende il minimo dei figli, la radice MAX il massimo.



I tre nodi MIN valgono $\min(3, 12, 8) = 3$, $\min(2, 4, 6) = 2$, $\min(14, 5, 2) = 2$; la radice vale $\max(3, 2, 2) = 3$. MAX sceglie quindi la prima mossa.

1.5.5 I limiti di Minimax

Minimax nella versione base esplora *tutto* l'albero. Già per il tris l'albero completo ha 549 946 nodi (partendo dalla scacchiera vuota). Molti rami però sono inutili: se un ramo è già sicuramente peggiore di un'alternativa trovata prima, non serve esplorarlo.

1.5.6 Potatura Alpha-Beta

Si tengono traccia di due valori, i migliori trovati finora lungo il percorso:

- α: il valore migliore (più *alto*) che **MAX** può garantirsi al livello corrente o sopra;
- β: il valore migliore (più *basso*) che **MIN** può garantirsi al livello corrente o sopra.

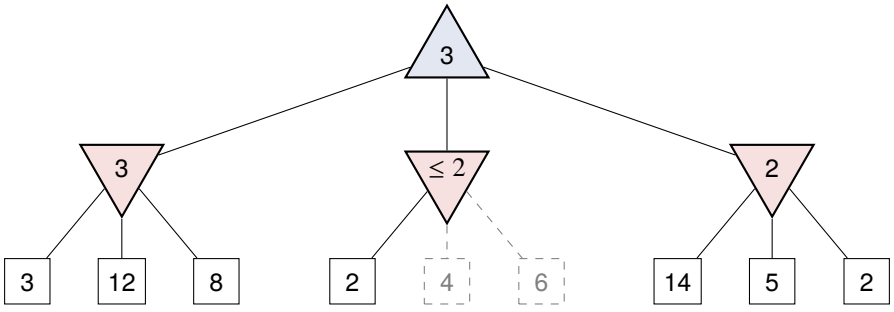
All'inizio $\alpha = -\infty$ e $\beta = +\infty$. Appena $\alpha \geq \beta$, il nodo corrente non potrà mai essere raggiunto con gioco ottimale e i figli restanti si possono **potare** (`break`).

```
def minimax(board, player, alpha, beta):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN, alpha, beta)
            best = max(val, best)
            alpha = max(best, alpha)
            if alpha >= beta:
                break
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX, alpha, beta)
            best = min(val, best)
            beta = min(best, beta)
            if alpha >= beta:
                break
        return best
```

Approfondimento (alpha-beta sull'esempio numerico). Riprendiamo l'albero di prima, esplorando i figli da sinistra a destra.

1. Primo nodo MIN ($\alpha = -\infty, \beta = +\infty$): vede 3, 12, 8 e restituisce 3. La radice aggiorna $\alpha = 3$.
2. Secondo nodo MIN ($\alpha = 3$): la prima foglia vale 2, quindi $\beta = 2$. Ora $\alpha = 3 \geq \beta = 2$: **potatura**. Le foglie 4 e 6 non vengono visitate. *Perché è corretto*: questo nodo MIN varrà al massimo 2, ma MAX ha già un'alternativa che vale 3, quindi non sceglierà mai questo ramo.
3. Terzo nodo MIN ($\alpha = 3$): vede 14 ($\beta = 14$), 5 ($\beta = 5$), 2 ($\beta = 2$) e restituisce 2.

Il risultato è 3, identico a Minimax, ma con due foglie in meno. Il risultato di alpha-beta è *sempre* identico a quello di Minimax: cambia solo quanto lavoro si fa.



Osservazione (l'ordine delle mosse conta). Quanto si pota dipende dall'ordine in cui si esplorano i figli. Se le mosse migliori vengono esplorate per prime, α e β si stringono subito e si pota molto di più. Nel caso ideale alpha-beta visita circa $b^{m/2}$ nodi invece di b^m (b = numero di mosse per stato, m = profondità): a parità di tempo si può cercare circa due volte più in profondità.

1.5.7 Oltre alpha-beta

Alpha-beta può ridurre l'albero di oltre il 99%, ma alcuni giochi restano troppo grandi (scacchi, go, . . .). Tecniche usate in pratica:

- **ricerca a profondità fissa** / *iterative deepening*: ci si ferma a una certa profondità, aumentandola finché c'è tempo;
- **valutazione euristica** degli stati: una stima del valore di stati non terminali;
- *quiescence search*: si continua a cercare oltre il limite solo nelle posizioni "instabili" (es. in mezzo a uno scambio di pezzi);
- *hashing* e *transposition tables*: si memorizza il valore di stati già visti, raggiungibili con sequenze di mosse diverse;
- *move ordering*: si provano prima le mosse promettenti, per potare di più;
- **basi di conoscenza**: aperture e tabelle dei finali negli scacchi.

1.6 Laboratorio: Minimax per il tris

LAB

[Lab_Minimax_blank.ipynb]

Obiettivo: implementare Minimax (e poi alpha-beta) per il tris, farlo giocare contro sé stesso e contro un umano. Il codice qui sotto è una soluzione completa ed è stata verificata: le asserzioni del notebook passano e le due IA pareggiano sempre.

1.6.1 Rappresentazione e condizioni di vittoria

La scacchiera è una lista di liste 3×3: `0` = vuota, `1` = X, `2` = O. `check_winner` restituisce `1` o `2` se quel giocatore ha un tris, altrimenti `0`. `is_full` dice se non ci sono celle vuote.

```python
def check_winner(board):
    lines = []
    lines.extend(board)                                  # righe
    lines.extend([[board[i][j] for i in range(3)]
                  for j in range(3)])                    # colonne
    lines.append([board[i][i] for i in range(3)])        # diagonale
    lines.append([board[i][2 - i] for i in range(3)])    # antidiagonale
    for line in lines:
        if line[0] != 0 and line[0] == line[1] == line[2]:
            return line[0]
    return 0

def is_full(board):
    return all(cell != 0 for row in board for cell in row)
```

> **Attenzione (la cella vuota).** Il controllo `line[0] != 0` è essenziale: senza, una riga vuota `[0,0,0]` verrebbe vista come "tre uguali" e la funzione restituirebbe un vincitore inesistente.

1.6.2 Minimax

X è MAX, O è MIN. La funzione `score` traduce il vincitore nel punteggio $+1/-1/0$. Invece di copiare la scacchiera a ogni mossa, si *modifica sul posto* e poi si **annulla la mossa** (*rollback*) prima di provare la successiva.

```python
from math import inf

def score(board):
    w = check_winner(board)
    if w == 1:
        return 1       # vince X (MAX)
    if w == 2:
        return -1      # vince O (MIN)
    return 0           # pareggio

def minimax(board, is_maximizing):
    if check_winner(board) or is_full(board):   # nodo foglia
```

```
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1                          # prova la mossa
                    best = max(best, minimax(board, False))
                    board[i][j] = 0                          # annulla la mossa
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax(board, True))
                    board[i][j] = 0
        return best
```

Attenzione (pseudocodice del notebook). Nel notebook, nel ramo MAX, il `return best` è indentato *dentro* il ciclo `for`: così la funzione restituirebbe il valore dopo aver provato solo la prima mossa. Il `return` va dopo il ciclo, come nelle slide e nel codice qui sopra.

1.6.3 Scegliere la mossa migliore

`minimax` restituisce un *valore*, non una mossa. `find_best_move` prova ogni mossa legale, valuta la posizione risultante con `minimax` e tiene quella con il valore più alto (se muove il giocatore 1) o più basso (se muove il giocatore 2).

```
def find_best_move(board, player):
    move = (-1, -1)                      # nessuna mossa se la scacchiera è piena
    best = -inf if player == 1 else inf
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                board[i][j] = player
                # dopo la mossa di 'player' tocca all'avversario:
                # se ha mosso X (1) ora minimizza O, e viceversa
                val = minimax(board, player == 2)
                board[i][j] = 0
                if (player == 1 and val > best) or (player == 2 and val < best):
                    best, move = val, (i, j)
    return move
```

Attenzione (errore tipico). Dopo aver piazzato la mossa di `player` bisogna chiamare `minimax` con il turno dell'*avversario*. Chiamarla con `is_maximizing = (player == 1)` è un errore comune: l'IA valuterebbe le posizioni come se toccasse di nuovo a lei.

1.6.4 Minimax contro sé stesso

Il notebook fornisce il ciclo di gioco: i giocatori si alternano chiamando `find_best_move` finché qualcuno vince o la scacchiera è piena. Con due giocatori ottimali la partita finisce **sempre in pareggio**. Con l'implementazione sopra la partita è:

$$(0,0),\ (1,1),\ (0,1),\ (0,2),\ (2,0),\ (1,0),\ (1,2),\ (2,1),\ (2,2) \quad \Rightarrow \quad \text{pareggio}$$

X	X	O
O	O	X
X	O	X

> **Approfondimento (perché il pareggio).** In ogni gioco combinatorio finito si può calcolare il valore di ogni stato risalendo dalle foglie (**induzione all'indietro**, *backward induction*), che è esattamente ciò che fa Minimax. Ne segue che uno dei due giocatori ha una strategia che garantisce la vittoria, oppure entrambi possono garantirsi almeno il pareggio (teorema di Zermelo). Per il tris vale la seconda: il valore minimax della scacchiera vuota è 0.

1.6.5 Minimax con potatura alpha-beta

Si aggiungono i parametri `alpha` e `beta` e il controllo di potatura dopo ogni figlio. Il resto del codice (incluso `find_best_move`) non cambia, salvo chiamare `minimax_ab` al posto di `minimax`.

```python
def minimax_ab(board, is_maximizing, alpha=-inf, beta=inf):
    if check_winner(board) or is_full(board):
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1
                    best = max(best, minimax_ab(board, False, alpha, beta))
                    board[i][j] = 0
                    alpha = max(alpha, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax_ab(board, True, alpha, beta))
                    board[i][j] = 0
                    beta = min(beta, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
```

Attenzione (`break` con due cicli annidati). Qui le mosse si scorrono con *due* cicli (`i` e `j`). Un `break` uscirebbe solo dal ciclo interno e l'esterno continuerebbe a esplorare le righe successive, annullando parte della potatura. Per questo si usa `return best` (va bene perché la mossa è già stata annullata). In alternativa si può prima costruire la lista delle celle vuote e usare un solo ciclo.

Risultati misurati. Stessa partita (pareggio), stesse mosse, ma molto meno lavoro:

	Minimax	Alpha-beta
Nodi visitati dalla scacchiera vuota	549 946	18 297
Nodi visitati in un'intera partita IA contro IA	618 175	36 684
Tempo della partita (Python)	circa 2,9 s	circa 0,17 s

Alpha-beta visita circa il 3% dei nodi di Minimax, cioè ne pota circa il 97%.

1.6.6 Giocare contro l'IA

L'ultima cella del notebook permette di sfidare l'algoritmo inserendo le mosse come `riga colonna` (indici da 0). Se l'implementazione è corretta **non si può mai vincere**: al massimo si pareggia.

Riepilogo della lezione

- Teoria dei giochi = decisioni tra più agenti razionali; AGT = progettare algoritmi in ambienti strategici.
- Aree del corso: giochi non cooperativi (equilibrio di Nash), coalizionali (valore di Shapley), scelta sociale (paradosso di Condorcet), mechanism design, matching.
- Dilemma del prigioniero: efficienza (Pareto) e stabilità (Nash) possono non coincidere.
- Giochi combinatori: 2 giocatori, informazione perfetta, niente caso, finiti, a turni, a somma zero. Si rappresentano con l'albero di gioco.
- Minimax: valore di uno stato = max (MAX) o min (MIN) dei valori dei figli.
- Alpha-beta: stesso risultato, pota i rami con $\alpha \geq \beta$. Nel tris visita 18 297 nodi invece di 549 946.

Prossima lezione: teoria dell'utilità e decisioni in condizioni di incertezza; giochi strategici e formalizzazione della *best response*.

LEZIONE 2

Teoria dell'utilità

TEORIA

Nella lezione precedente le utilità erano numeri dati ($+1$, 0, -1 per il tris). Qui si fa il passo indietro: *da dove vengono* quei numeri? La risposta è che non sono un dato primitivo, ma una **rappresentazione** delle preferenze del giocatore. Il percorso della lezione è:

1. concetti di base: alternative e preferenze;
2. rappresentare preferenze ordinali (utilità **ordinale**);
3. oltre l'ordinale: lotterie e utilità **cardinale** (VNM);
4. atteggiamento verso il rischio;
5. applicazioni e introduzione alla teoria delle decisioni.

2.1 Concetti di base

2.1.1 Alternative

Un agente sceglie all'interno di un insieme X di **alternative**, che devono essere:

- **mutuamente esclusive**: se ne realizza una sola;
- **esaustive**: coprono tutti i casi possibili.

Esempio (scelta dei corsi). Le opzioni sono {Deep Learning, Algorithmic Game Theory}, ma queste *non* sono le alternative: si possono seguire entrambi i corsi o nessuno. L'insieme corretto è

$$X = \{\,\mathrm{DL},\ \mathrm{AGT},\ \mathrm{DLAGT},\ \mathrm{N}\,\},$$

dove DLAGT = entrambi e N = nessuno dei due.

Osservazione (è un passo di modellazione, non una formalità). Costruire X *bene* è la prima cosa da fare in un modello. Con n opzioni combinabili liberamente le alternative sono 2^n: già qui si vede che la dimensione dello spazio delle alternative è un problema *computazionale*, non solo concettuale.

2.1.2 Preferenze

Definizione 2.1 (Relazione di preferenza). Le **preferenze** sono una relazione $\succsim$ su X, cioè un sottoinsieme di $X \times X$. Si legge $x \succsim y$ come "x è *almeno buono quanto* y". La relazione è:

- **completa** se $\forall x, y \in X$ vale $x \succsim y$ oppure $y \succsim x$;
- **transitiva** se $\forall x, y, z \in X$, $[x \succsim y \text{ e } y \succsim z] \Rightarrow x \succsim z$.

Una preferenza è una **relazione di preferenza** se e solo se è completa e transitiva.

Da $\succsim$ si derivano altre due relazioni:

$$\begin{aligned} \text{preferenza } \textbf{stretta}: &\qquad x \succ y \iff [\, x \succsim y \text{ e } y \not\succsim x \,]; \\ \textbf{indifferenza}: &\qquad x \sim y \iff [\, x \succsim y \text{ e } y \succsim x \,]. \end{aligned}$$

Osservazione (completezza non vuol dire "decidere sempre"). La completezza non dice che l'agente preferisce sempre qualcosa a qualcos'altro: dice che sa sempre *confrontare* due alternative. Il caso "mi sono indifferenti" è ammesso ed è proprio $x \sim y$. Quello che la completezza esclude è l'incomparabilità ("non so dire").

2.2 Rappresentare le preferenze: utilità ordinale

Definizione 2.2 (Funzione di utilità). Una relazione di preferenza $\succsim$ è **rappresentata** da una funzione di utilità $u : X \to \mathbb{R}$ nel senso seguente:

$$x \succsim y \iff u(x) \geq u(y) \qquad \forall x, y \in X.$$

Se un giocatore trova x almeno buono quanto y, allora $u(x)$ deve essere almeno alto quanto $u(y)$. In questo senso il giocatore si comporta *come se* (*as if*) stesse massimizzando $u(\cdot)$.

Osservazione (la direzione della spiegazione). L'ultima frase va presa alla lettera e sarà ripresa alla fine della lezione: l'utilità è un *modo di descrivere* le preferenze, non la loro causa. Non si sceglie x perché ha utilità più alta; ha utilità più alta perché è ciò che si sceglie.

Teorema 2.3 (Rappresentazione ordinale). Sia X *finito*. Una preferenza può essere rappresentata da una funzione di utilità se e solo se è completa e transitiva. Inoltre, se $u : X \to \mathbb{R}$ rappresenta $\succsim$ e $f : \mathbb{R} \to \mathbb{R}$ è **strettamente crescente**, allora anche $f \circ u$ rappresenta $\succsim$.

Per l'ultima affermazione queste utilità si dicono **ordinali**: conta solo l'*informazione d'ordine*.

2.2.1 Le condizioni sono necessarie

Transitività. Sia $X = \{a, b, c\}$ e si supponga il ciclo $a \succ b \succ c \succ a$. Se esistesse u si avrebbe $u(a) > u(b) > u(c) > u(a)$, cioè $u(a) > u(a)$: assurdo.

Approfondimento (la *money pump*). Filosofi ed economisti direbbero che violare la transitività genera una **pompa di soldi**. Se l'agente ha il ciclo sopra e possiede c, allora preferisce strettamente b a c: gli si può vendere b in cambio di c più un piccolo importo ε. Poi preferisce a a b: altro scambio, altro ε. Poi preferisce c ad a: terzo scambio. L'agente è tornato al punto di partenza avendo pagato 3ε, e il ciclo si può ripetere all'infinito. È l'argomento "pratico" a favore della transitività: chi la viola si fa svuotare le tasche.

Completezza. Si assuma di avere una funzione di utilità $u : X \to \mathbb{R}$. Dati due numeri $u(x)$ e $u(y)$, sono ammissibili solo due possibilità: $u(x) \geq u(y)$ oppure $u(y) \geq u(x)$. Quindi $x \succsim y$ o $y \succsim x$: la completezza è una necessità matematica, ereditata dal fatto che l'ordine $\geq$ su $\mathbb{R}$ è totale.

2.2.2 Le condizioni sono sufficienti

Dimostrazione del teorema. Sia $X = \{x_1, \ldots, x_n\}$. Grazie a completezza e transitività si possono partizionare gli elementi di X in k **classi di indifferenza** $C_1, \ldots, C_k$ (ogni classe raccoglie alternative tra loro indifferenti) ordinate in modo che $C_1 \succ C_2 \succ \ldots \succ C_k$. Si definisce allora

$$u(x) = k - i + 1 \qquad \text{per ogni } x \in C_i,$$

cioè $u = k$ sulla classe migliore, $u = k - 1$ sulla successiva, fino a $u = 1$ su C_k. Per costruzione $u(x) \geq u(y) \iff x \succsim y$. □

Attenzione (dove serve che X sia finito). L'ipotesi "X finito" non è decorativa: serve a poter costruire le classi di indifferenza e a numerarle. Con X infinito completezza e transitività *non* bastano. Il controesempio classico sono le preferenze **lessicografiche** su $X = \mathbb{R}^2$, dove $(a_1, a_2) \succsim (b_1, b_2)$ se $a_1 > b_1$, oppure se $a_1 = b_1$ e $a_2 \geq b_2$: la relazione è completa e transitiva ma non ammette alcuna rappresentazione con una u a valori reali.

2.2.3 Esempi: l'utilità ordinale non è interpretabile

Con $X = \{a, b, c\}$ e preferenze $a \succ b \succ c$, tutte e tre queste funzioni rappresentano le *stesse* preferenze:

	u_1	u_2	u_3
$u(a)$	-1	30	90
$u(b)$	-2	20	2
$u(c)$	-3	10	1

Il valore numerico dell'utilità ordinale **non è interpretabile**: non dice con quale *intensità* un giocatore preferisce a a b. Con u_2 le differenze sono uguali, con u_3 il salto tra a e b sembra enorme, ma le preferenze rappresentate sono identiche.

Osservazione (perché non basta). Finché si sceglie tra alternative *certe*, l'ordine basta: si prende la migliore. Ma appena entra in gioco l'**incertezza** (una scelta dà a con probabilità 0,5 e c con probabilità 0,5) serve sapere *quanto* b sta tra a e c: serve un'utilità **cardinale**. È il problema che risolve la teoria di Von Neumann–Morgenstern.

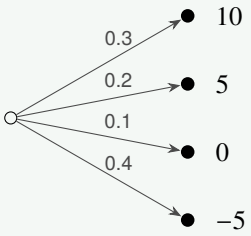
2.3 Oltre l'ordinale: le lotterie

Definizione 2.4 (Lotteria semplice). Una **lotteria semplice** è una tupla

$$L = (p_1, x_1;\ p_2, x_2;\ \ldots;\ p_n, x_n)$$

dove $x_1, \ldots, x_n \in X \subseteq \mathbb{R}$ sono **premi monetari** e $(p_1, \ldots, p_n)$ è una distribuzione di probabilità ($p_i \geq 0$, $\sum_i p_i = 1$). Si indica con $\mathcal{L}$ l'insieme delle lotterie semplici.

Esempio (una lotteria). $L = (0{,}3,\ 10;\ 0{,}2,\ 5;\ 0{,}1,\ 0;\ 0{,}4,\ -5)$.



2.3.1 Lotterie composte

Una lotteria può avere come premi altre lotterie. Siano

$$L_1 = (0{,}9:\ -1\$,\ 0{,}1:\ 5\$), \qquad L_2 = (0{,}75:\ -1\$,\ 0{,}25:\ 2\$).$$

La **lotteria composta** $L_c = (0{,}5:\ L_1,\ 0{,}5:\ L_2)$ equivale alla lotteria semplice che si ottiene moltiplicando le probabilità lungo i rami e sommando quelle che portano allo stesso premio:

$$L_e = \big(0{,}5\times0{,}9+0{,}5\times0{,}75:\ -1\$,\quad 0{,}5\times0{,}25:\ 2\$,\quad 0{,}5\times0{,}1:\ 5\$\big) = (0{,}825:\ -1\$,\ 0{,}125:\ 2\$,\ 0{,}05:\ 5\$).$$

Osservazione (ipotesi di riduzione). Questo passaggio è un'*assunzione* del modello: si suppone che al giocatore interessi solo la distribuzione finale sui premi, non il modo in cui ci si arriva (due estrazioni invece di una). Grazie a essa basta studiare $\mathcal{L}$, l'insieme delle lotterie *semplici*.

2.3.2 Valore atteso

Definizione 2.5 (Valore atteso). Per $L = (p_1, x_1; \ldots; p_n, x_n)$ il **valore atteso** è

$$\mathbb{E}(L) = \sum_{i=1}^{n} p_i\, x_i.$$

Per la lotteria dell'esempio:

$$\mathbb{E}(L) = 0{,}3 \times 10 + 0{,}2 \times 5 + 0{,}1 \times 0 + 0{,}4 \times (-5) = 2.$$

Giocando ripetutamente quella lotteria ci si aspetta di vincere in media 2.

Attenzione (il valore atteso non basta a decidere). Verrebbe da usare $\mathbb{E}$ come criterio di scelta, ma è sbagliato. Si consideri il **paradosso di San Pietroburgo**: si lancia una moneta finché non esce testa; se esce al lancio k si vincono 2^k euro. Il valore atteso è $\sum_{k\geq 1} 2^{-k} \cdot 2^k = \infty$, eppure nessuno pagherebbe più di poche decine di euro per giocare. Serve applicare l'utilità *ai premi*, non ai soldi.

2.3.3 Utilità attesa

Definizione 2.6 (Utilità attesa e funzione di Bernoulli). Data una relazione di preferenza $\succsim$ su $\mathcal{L}$, una funzione di utilità $U : \mathcal{L} \to \mathbb{R}$ è una **funzione di utilità attesa** se può essere scritta come

$$U(L) = \sum_{i=1}^{n} p_i \, u(x_i)$$

per qualche funzione $u : \mathbb{R} \to \mathbb{R}$. La funzione u si chiama **funzione di utilità di Bernoulli**.

Attenzione (due utilità diverse, due simboli diversi). u è definita sui *premi* (numeri di euro), U sulle *lotterie*. La forma di U è vincolata: deve essere **lineare nelle probabilità**. Non ogni U che rappresenta preferenze su $\mathcal{L}$ ha questa forma; il teorema VNM dice esattamente quando ce l'ha.

2.4 L'utilità di Von Neumann–Morgenstern

Von Neumann e Morgenstern (VNM) hanno fornito le condizioni sotto cui una relazione di preferenza $\succsim$ su $\mathcal{L}$ è rappresentabile da una funzione di utilità attesa.

Definizione 2.7 (Assiomi VNM).

Assioma 1 (**ordine di preferenza**) $\succsim$ è completa e transitiva.

Assioma 2 (**continuità**) se $L \succ M \succ N$, esiste $p \in [0,1]$ tale che $pL + (1-p)N \sim M$.

Assioma 3 (**indipendenza**) per ogni lotteria N e ogni $p \in [0,1]$,

$$L \succsim M \iff pL + (1-p)N \succsim pM + (1-p)N.$$

Osservazione (come leggere gli assiomi 2 e 3). La **continuità** dice che non esistono premi "infinitamente" migliori di altri: se M sta tra L e N, c'è una miscela di L e N esattamente equivalente a M. L'**indipendenza** dice che un esito *comune* alle due lotterie non deve cambiare il confronto: N compare con la stessa probabilità $1-p$ da entrambe le parti, quindi deve essere irrilevante. È l'assioma più forte dei tre e, come si vedrà con Allais, è quello che gli esseri umani violano.

Teorema 2.8 (Von Neumann–Morgenstern). Una relazione binaria $\succsim$ su $\mathcal{L}$ ammette una rappresentazione di utilità attesa se e solo se soddisfa gli assiomi 1–3. Inoltre, se U e V sono due rappresentazioni di utilità attesa di $\succsim$, allora esistono costanti $a, b \in \mathbb{R}$ con $a > 0$ tali che

$$U(\cdot) = a\,V(\cdot) + b.$$

L'ultima affermazione dice che la rappresentazione VNM è unica **a meno di trasformazioni affini**.

Osservazione (ordinale contro cardinale). È qui la differenza con il Teorema 1. Lì si poteva applicare a *u qualunque* funzione strettamente crescente; qui solo $ax + b$ con $a > 0$. Il margine di libertà è molto più stretto, e questo rende i numeri *interpretabili*: sono fissati l'origine e l'unità di misura, come per la temperatura in gradi Celsius. Ha senso dire che il salto da a a b è il doppio del salto da b a c; non ha senso dire che a vale il doppio di b. Si parla di **scala a intervalli**.

2.4.1 Necessità degli assiomi

Ordine di preferenza. Già dimostrato per il Teorema 1.

Continuità. Sia $L \succ M \succ N$; allora $u(L) > u(M) > u(N)$. Si ponga

$$p = \frac{u(M) - u(N)}{u(L) - u(N)} \in (0, 1).$$

Per la definizione di utilità attesa:

$$\begin{aligned} u\big(pL + (1-p)N\big) &= p\,u(L) + (1-p)\,u(N) \\ &= p\,u(L) + u(N) - p\,u(N) \\ &= p\big(u(L) - u(N)\big) + u(N) \\ &= \frac{u(M) - u(N)}{u(L) - u(N)}\big(u(L) - u(N)\big) + u(N) \\ &= u(M) - u(N) + u(N) = u(M). \end{aligned}$$

Quindi $pL + (1-p)N \sim M$: la continuità è necessaria.

Indipendenza. Siano $L_1 = pL + (1-p)N$ e $L_2 = pM + (1-p)N$. Per linearità dell'attesa:

$$\begin{aligned} u(L_1) &= p\,u(L) + (1-p)\,u(N), \\ u(L_2) &= p\,u(M) + (1-p)\,u(N). \end{aligned}$$

Sottraendo membro a membro, il termine comune in N si cancella:

$$u(L_1) - u(L_2) = p\big(u(L) - u(M)\big).$$

Se $L \succ M$ allora $u(L) > u(M)$, dunque $u(L_1) > u(L_2)$, cioè $L_1 \succ L_2$. L'indipendenza è necessaria.

Osservazione (da dove viene la "forza" dell'indipendenza). La cancellazione del termine $(1-p)u(N)$ funziona *perché* U è lineare nelle probabilità. Indipendenza e linearità sono in sostanza la stessa proprietà vista da due lati: uno comportamentale (l'esito comune non conta), l'altro algebrico.

2.4.2 Sufficienza: dimostrazione del teorema VNM

La dimostrazione si divide in due parti:

- **Parte 1**: si mostra che $U(L) = \sum_i p_i\,u(x_i)$;
- **Parte 2**: si mostra che $L \succ M \iff U(L) > U(M)$ per ogni $L, M \in \mathcal{L}$.

Parte 1. Si supponga di avere n esiti $o_1, \dots, o_n$. Per completezza e transitività si possono ordinare dal peggiore al migliore: $o_1 \precsim \dots \precsim o_n$ (nel senso o_n migliore). Si fissano i due estremi della scala:

$$u(o_1) = 0, \qquad u(o_n) = 1.$$

Si definisce la famiglia di lotterie semplici

$$L(p) = p \cdot o_n + (1 - p) \cdot o_1,$$

cioè "il premio migliore con probabilità p, il peggiore con probabilità $1 - p$". Per l'assioma di continuità, per ogni esito o_i esiste una probabilità $q_i \in [0, 1]$ tale che $L(q_i) \sim o_i$; si pone $u(o_i) = q_i$. Ne segue che l'utilità di una lotteria $M = \sum_i p_i o_i$ è l'attesa di u:

$$u(M) = u\Big(\sum_i p_i o_i\Big) = \sum_i p_i \, u(o_i) = \sum_i p_i \, q_i.$$

> **Osservazione (che cosa è davvero q_i).** q_i è un modo operativo di *misurare* l'utilità: è la probabilità di vincere il premio migliore che rende il giocatore indifferente a ricevere o_i con certezza. È una definizione che si può realizzare come esperimento, chiedendo alla persona di aggiustare q finché non è indifferente, ed è esattamente questo che rende la scala cardinale invece che ordinale: q_i posiziona o_i *tra* i due estremi.

Parte 2. Per l'assioma di continuità il giocatore è indifferente tra l'esito o_i e la lotteria $q_i \cdot o_n + (1 - q_i) \cdot o_1$. Insieme all'assioma di indipendenza (si sostituisce un esito alla volta dentro la lotteria) il giocatore è indifferente tra le due lotterie

$$M = \sum_i p_i o_i \qquad \text{e} \qquad M' = \sum_i p_i \big[\, q_i \cdot o_n + (1 - q_i) \cdot o_1 \,\big].$$

Ma M' coinvolge solo i due esiti estremi, quindi si può riscrivere come lotteria semplice:

$$M' = \Big(\sum_i p_i q_i\Big) \cdot o_n + \Big(\sum_i p_i (1 - q_i)\Big) \cdot o_1 = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Ogni lotteria è dunque equivalente a una lotteria sui soli due estremi, la cui unica caratteristica è la probabilità $U(M)$ di ottenere il premio migliore.

Si supponga ora $L \succ M$ e si definiscano

$$L' = U(L) \cdot o_n + \big(1 - U(L)\big) \cdot o_1,$$
$$M' = U(M) \cdot o_n + \big(1 - U(M)\big) \cdot o_1.$$

Vale l'ordine $L' \sim L \succ M \sim M'$, quindi $L' \succ M'$. Ma L' e M' differiscono solo per la probabilità del premio migliore, e tra due lotterie di questa famiglia si preferisce quella con la probabilità più alta: dunque $U(L) > U(M)$. Vale anche il viceversa, quindi $L \succ M \iff U(L) > U(M)$. □

2.5 Il paradosso di Allais

Si risponda d'istinto a due domande. *Prima*: quale lotteria si preferisce?

$$L = (1\text{: } 1.000.000\$), \qquad M = (0{,}89\text{: } 1.000.000\$,\ \ 0{,}10\text{: } 5.000.000\$,\ \ 0{,}01\text{: } 0\$).$$

Seconda: e tra queste?

$$N = (0{,}89\colon 0\$,\ \ 0{,}11\colon 1.000.000\$), \qquad O = (0{,}90\colon 0\$,\ \ 0{,}10\colon 5.000.000\$).$$

La maggior parte delle persone sceglie L nella prima coppia (il milione sicuro) e O nella seconda. Questa combinazione è **incompatibile** con la teoria dell'utilità attesa.

Approfondimento (perché è una contraddizione). Si normalizzi $u(0) = 0$ e si scrivano $u_1 = u(1\text{M})$, $u_5 = u(5\text{M})$.

- $L \succ M$ significa $u_1 > 0{,}89\,u_1 + 0{,}10\,u_5$, cioè $0{,}11\,u_1 > 0{,}10\,u_5$;
- $O \succ N$ significa $0{,}10\,u_5 > 0{,}11\,u_1$.

Le due disuguaglianze si contraddicono: *nessuna* funzione di utilità u può giustificare entrambe le scelte.
Visto con l'assioma di indipendenza è ancora più netto. Si scriva ogni lotteria come miscela con $p = 0{,}11$:

	parte con probabilità 0,11	parte con probabilità 0,89
L	1M certo	1M
M	$(\frac{10}{11}\colon 5\text{M},\ \frac{1}{11}\colon 0)$	1M
N	1M certo	0
O	$(\frac{10}{11}\colon 5\text{M},\ \frac{1}{11}\colon 0)$	0

Le coppie (L, M) e (N, O) differiscono nella prima colonna esattamente allo stesso modo; la seconda colonna è un esito *comune* (dentro ogni coppia). Per l'indipendenza l'esito comune non deve contare, quindi chi preferisce L a M deve preferire N a O.

Osservazione (che cosa insegna il paradosso). Allais non dimostra che la teoria VNM è sbagliata: dimostra che è **normativa** e non **descrittiva**. Dice come si *dovrebbe* decidere per essere coerenti, non come le persone decidono davvero. L'effetto in gioco si chiama *certainty effect*: il salto da 0,99 a 1 (certezza) pesa psicologicamente molto più del salto da 0,10 a 0,11. Le teorie descrittive alternative (per esempio la *prospect theory* di Kahneman e Tversky) nascono proprio da qui.

2.6 Atteggiamento verso il rischio

Definizione 2.9 (Lotteria equa). Una lotteria $L = (p\colon x,\ (1-p)\colon y)$ è **equa** se il suo valore atteso è nullo:

$$\mathbb{E}(L) = p\,x + (1-p)\,y = 0.$$

Attenzione (notazione delle slide). Le slide scrivono "$L = p \cdot x + (1-p) \cdot y = 0$". La lotteria L è un oggetto (una distribuzione), non un numero: a essere nullo è il suo *valore atteso* $\mathbb{E}(L)$. Si tratta di un abuso di notazione, ma la sostanza è "giocare questa lotteria non costa e non rende nulla, in media".

Definizione 2.10 (Atteggiamenti verso il rischio). Un giocatore con funzione di utilità di Bernoulli u è:

- **neutrale al rischio** se u è **lineare**, $u(x) = ax + b$ (con $a > 0$): è indifferente di fronte alle lotterie eque;
- **avverso al rischio** se u è **concava**: se strettamente concava, non gioca *nessuna* lotteria equa;
- **amante del rischio** (*risk seeking*) se u è **convessa**: se strettamente convessa, gioca *tutte* le lotterie eque.

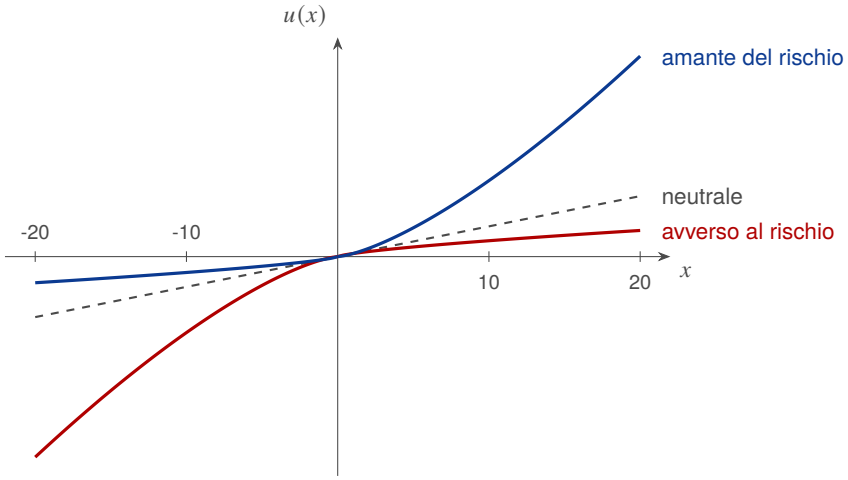


Figura 2.1: Le tre forme della funzione di utilità sui guadagni. L'avverso al rischio soffre una perdita più di quanto goda un guadagno della stessa entità; per l'amante del rischio vale il contrario.

Approfondimento (perché concavità significa avversione al rischio). È la **disuguaglianza di Jensen**. Se u è concava, per ogni lotteria L

$$\mathbb{E}\big[u(L)\big] \;\leq\; u\big(\mathbb{E}[L]\big),$$

cioè l'utilità del gioco è al più l'utilità di incassare con certezza il valore atteso. Chi è avverso al rischio preferisce quindi sempre il certo all'aleatorio di pari media; per una lotteria equa aggiunta alla propria ricchezza, non giocare è almeno altrettanto buono che giocare.
Si definisce **equivalente certo** (*certainty equivalent*) $CE(L)$ il premio certo che dà la stessa utilità della lotteria, cioè $u(CE(L)) = \mathbb{E}[u(L)]$. La differenza

$$\mathbb{E}[L] - CE(L)$$

è il **premio per il rischio**: quanto il giocatore è disposto a rinunciare per eliminare l'incertezza. Per un avverso al rischio è positivo, ed è esattamente la quantità su cui si regge il mercato assicurativo.

2.7 Applicazioni

In entrambe le applicazioni la funzione di Bernoulli è $u(x) = \sqrt{x}$, che è strettamente concava: il giocatore è avverso al rischio.

2.7.1 Condivisione del rischio (*risk sharing*)

Due giocatori avversi al rischio, con $u(x) = \sqrt{x}$, possiedono due attività rischiose *indipendenti*

$$A_1 = A_2 = (0{,}5,\ 100;\ 0{,}5,\ 0).$$

L'utilità di ciascuna attività, presa da sola, è

$$u(A_1) = u(A_2) = 0{,}5 \times \sqrt{100} + 0{,}5 \times \sqrt{0} = 5.$$

Se i due giocatori formano un **fondo comune** in cui ciascuno possiede metà del totale, ogni giocatore si ritrova con l'attività

$$A_m = (0{,}25,\ 100;\ 0{,}5,\ 50;\ 0{,}25,\ 0),$$

la cui utilità è

$$u(A_m) = 0{,}25 \times \sqrt{100} + 0{,}5 \times \sqrt{50} + 0{,}25 \times \sqrt{0} \approx 2{,}5 + 3{,}54 = 6{,}04 > 5.$$

Entrambi i giocatori stanno **meglio** pur non avendo cambiato il valore atteso.

> **Approfondimento (da dove viene A_m e perché funziona).** Le due attività sono indipendenti, quindi i quattro casi (vince A_1 sì/no, A_2 sì/no) hanno probabilità 0,25 ciascuno. Il montante totale è 200, 100, 100, 0; metà del fondo vale quindi 100, 50, 50, 0, da cui $A_m = (0{,}25, 100;\ 0{,}5, 50;\ 0{,}25, 0)$. Il valore atteso è rimasto 50 in entrambi i casi: quello che è cambiato è la *varianza*, scesa da 2500 a 1250. Mettere insieme rischi indipendenti li **diversifica**, e per un giocatore con u concava meno varianza a parità di media significa più utilità. È il principio su cui si reggono i fondi comuni di investimento e le mutue assicurative.

2.7.2 Assicurazione

Un giocatore avverso al rischio con $u(x) = \sqrt{x}$ possiede l'attività $A = (0{,}5,\ 100;\ 0{,}5,\ 0)$. Esiste una compagnia di assicurazione **neutrale al rischio** con molto capitale.

Quanto è disposto a pagare il giocatore? Il giocatore paga un premio P e in cambio riceve con certezza 100; il suo patrimonio certo è quindi $100 - P$. Accetta se questo gli dà almeno l'utilità della lotteria:

$$u(100 - P) \ \geq\ 0{,}5 \times u(100) + 0{,}5 \times u(0) \implies \sqrt{100 - P} \geq 5 \implies 100 - P \geq 25 \implies P \leq 75.$$

Quanto vuole la compagnia? Essendo neutrale al rischio, le basta che il premio copra il valore atteso del risarcimento:

$$P \ \geq\ 0{,}5 \times 100 + 0{,}5 \times 0 \implies P \geq 50.$$

Conclusione. Entrambe le parti guadagnano se il contratto si stipula a un premio

$$P \in [50,\ 75].$$

> **Osservazione (lettura con l'equivalente certo).** $\sqrt{100 - P} \geq 5$ significa $100 - P \geq 25$: l'equivalente certo della lotteria è $CE(A) = 25$, mentre il suo valore atteso è $\mathbb{E}(A) = 50$. Il premio per

il rischio è $50 - 25 = 25$, ed è esattamente l'ampiezza dell'intervallo $[50, 75]$: il **surplus** che le due parti si dividono. L'assicurazione crea valore perché sposta il rischio da chi lo odia a chi gli è indifferente.

Attenzione (il patrimonio iniziale conta). Qui si è assunto implicitamente che il giocatore non abbia altra ricchezza: l'utilità si calcola su $100 - P$ e su 100 o 0. Se avesse un patrimonio iniziale w, le utilità sarebbero $u(w + 100 - P)$ e $u(w + 100)$, $u(w)$, e il premio massimo accettabile sarebbe diverso (più basso in termini relativi: più si è ricchi, meno si è avversi al rischio con $u(x) = \sqrt{x}$). La funzione di Bernoulli si applica alla *ricchezza finale*, non al guadagno della singola scommessa.

2.8 Riepilogo sulla teoria VNM

- La teoria VNM fornisce un modo per costruire funzioni di utilità **cardinali**: i valori numerici sono interpretabili su una **scala a intervalli** e, in certe situazioni, su una scala di rapporti.
- La teoria VNM fornisce una teoria **normativa** della decisione: massimizzare l'utilità attesa. È molto usata in applicazioni finanziarie ed economiche.
- La teoria VNM **non descrive** il comportamento umano. Un'alternativa non è preferita perché ha un'utilità più alta; piuttosto, le si associa un'utilità più alta perché è l'alternativa preferita.

2.9 Laboratorio: Battaglia navale e ricerca bayesiana

LAB

[Battleship_Blank.ipynb]

Obiettivo: costruire un'IA per la **battaglia navale**, partendo da strategie euristiche semplici e arrivando a una strategia **adattiva** che a ogni turno spara dove l'*utilità attesa* è massima. Quest'ultima tecnica si chiama **ricerca bayesiana** (*Bayesian search*) ed è uno degli algoritmi più efficaci per i problemi di ricerca e di "caccia al tesoro".

Il collegamento con la teoria della lezione è diretto: non c'è un avversario strategico, ma c'è **incertezza**. Si deve scegliere l'azione (la cella su cui sparare) che massimizza l'utilità attesa, dove la probabilità viene aggiornata a ogni colpo con le informazioni raccolte.

2.9.1 Regole e impostazione

Il campo è una griglia 10×10 su cui sono disposte cinque navi, in orizzontale o in verticale, senza sovrapposizioni e senza uscire dal bordo:

Nave	Celle
Portaerei (*Carrier*)	5
Corazzata (*Battleship*)	4
Incrociatore (*Cruiser*)	3
Sommergibile (*Submarine*)	3
Cacciatorpediniere (*Destroyer*)	2
Totale	**17**

Non essendoci un avversario vero, le strategie non si valutano in partite vinte ma in **numero di turni necessari ad affondare tutte le navi**: 17 è il minimo teorico (nessun colpo a vuoto), 100 il massimo.

Lo stato della conoscenza si tiene in una matrice `shot_grid` con `0` = cella inesplorata, `1` = colpo a vuoto, `2` = colpo a segno.

> **Attenzione (un bug nel generatore fornito).** La funzione `create_random_disposition` del notebook estrae le coordinate così:
>
> ```
> if horizontal:
>     x = np.random.randint(0, grid_size - 1)        # riga
>     y = np.random.randint(0, grid_size - length)   # colonna di partenza
> else:
>     x = np.random.randint(0, grid_size - length)
>     y = np.random.randint(0, grid_size - 1)
> ```
>
> `np.random.randint(0, n)` esclude `n`. Quindi per una nave orizzontale la riga sta in $[0, 8]$ (la riga 9 non esce mai) e la colonna di partenza in $[0,\ 9 - \texttt{length}]$, cioè l'ultima cella occupata arriva al più alla colonna 8. Simmetricamente per le navi verticali. Il risultato è che **l'ultima riga e l'ultima colonna non sono mai occupate**: su 2000 partite generate, le celle della riga 9 e della colonna 9 risultano vuote in tutte e 2000 (contro circa 2800 occupazioni per la riga 0 e per la colonna 0).

Gli estremi corretti sarebbero `randint(0, grid_size)` e `randint(0, grid_size - length + 1)`.
Conseguenza pratica: le cifre misurate sotto sono un po' ottimistiche, perché il 19% del campo è di fatto acqua garantita. Il confronto *tra* le strategie resta però valido, dato che tutte giocano sulle stesse partite.

2.9.2 Strategia 1: tiro casuale

La strategia più semplice: si spara a caso, ignorando del tutto gli esiti. Non è una vera strategia, ma serve da **riferimento** (*baseline*): battere il caso è il minimo sindacale.

```python
def random_attack_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    cells = [(i, j) for i in range(10) for j in range(10)]
    np.random.shuffle(cells)
    hits = turns = 0
    for (i, j) in cells:
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            if hits == 17:              # tutte le navi affondate
                break
    return shot_grid, turns
```

Osservazione (mescolare invece di estrarre). Sparare "a caso" si può implementare estraendo una cella e riprovando se è già stata colpita, ma verso la fine della partita quasi tutte le celle sono esplorate e il ciclo di riprova diventa lento. Mescolare una volta la lista delle 100 celle e scorrerla dà la stessa distribuzione senza riprove.

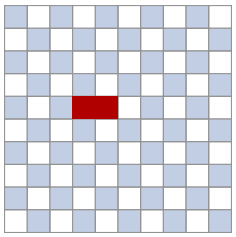
2.9.3 Strategia 2: *hunt and target*

La strategia casuale va male perché tratta tutte le celle come equiprobabili e *butta via* l'informazione dei colpi a segno. L'idea di **hunt and target** è alternare due modalità:

- **hunt** (caccia): si cerca a caso finché non si colpisce qualcosa;
- **target** (bersaglio): appena si colpisce, si spara nelle celle adiacenti (sopra, sotto, destra, sinistra) non ancora esplorate, finché la nave è affondata.

È il classico compromesso tra **esplorazione** e **sfruttamento**.

In più, in modalità hunt non serve sparare ovunque: la nave più piccola occupa 2 celle, quindi una nave non può nascondersi dentro una sola "casella nera" di una **scacchiera**. Basta sparare sulle celle con $(i + j)$ pari: 50 celle invece di 100, con la garanzia di intercettare comunque ogni nave.



Le celle della scacchiera (in blu) bastano: qualunque nave da 2 celle (in rosso) ne tocca almeno una.

```python
def hunt_and_target_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    hunt = [(i, j) for i in range(10) for j in range(10) if (i + j) % 2 == 0]
    np.random.shuffle(hunt)
    targets = []                        # pila delle celle da verificare
    hits = turns = 0
    while hits < 17:
        if targets:
            i, j = targets.pop()
        else:
            i, j = hunt.pop()
        if shot_grid[i, j]:             # gia' esplorata: salta il turno
            continue
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
            hits += 1
            for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                a, b = i + di, j + dj
                if 0 <= a < 10 and 0 <= b < 10 and shot_grid[a, b] == 0:
                    targets.append((a, b))
```

Attenzione (la pila può contenere celle già colpite). Una cella può finire nella pila `targets` due volte (da due colpi a segno diversi), oppure essere colpita nel frattempo. Il controllo `if shot_grid[i, j]: continue` prima di sparare è indispensabile: senza, si conterebbero turni per colpi mai sparati e il risultato sarebbe gonfiato.

Approfondimento (si può fare meglio in modalità target). L'implementazione sopra è "la più semplice": spara alle quattro celle adiacenti senza ragionare. Un miglioramento classico è sfruttare l'**orientamento**: dopo due colpi a segno allineati si sa che la nave è orizzontale o verticale, quindi si possono scartare le celle adiacenti nell'altra direzione. La ricerca bayesiana della prossima sezione ottiene questo effetto *automaticamente*, senza regole scritte a mano.

2.9.4 Strategia 3: ricerca bayesiana

Qui si usa davvero la teoria della lezione. A ogni turno si costruisce una **distribuzione a posteriori** sulla posizione delle navi e si spara dove la probabilità di colpire è massima, cioè dove è massima l'**utilità attesa** (con utilità 1 per il colpo a segno e 0 per il colpo a vuoto, l'utilità attesa di una cella *è* la sua probabilità di contenere una nave).

Il conteggio è semplice: per ogni nave rimasta e per ogni sua possibile posa (orizzontale o verticale) *compatibile* con quanto si sa, si aggiunge un voto a ciascuna cella coperta. Una posa è compatibile se non copre nessuna cella risultata a vuoto. Si distinguono due regimi:

- nessun colpo a segno "aperto" (**hunt**): si contano tutte le pose compatibili;
- ci sono colpi a segno (**target**): si contano solo le pose che *spiegano* almeno un colpo a segno, pesandole con $10^{(\text{colpi coperti})}$ per premiare le pose che ne spiegano di più.

In entrambi i casi i voti si assegnano solo alle celle ancora **inesplorate**: sparare dove si è già sparato non ha alcun valore.

```python
SIZES = [5, 4, 3, 3, 2]

def update_posterior(shot_map, sizes):
    post = conta_pose(shot_map, sizes, targeting=(shot_map == 2).any())
    if post.max() == 0:                    # nessuna posa spiega i colpi a segno
        post = conta_pose(shot_map, sizes, targeting=False)
    if post.max() == 0:                    # nessuna posa compatibile: cella libera
        post = (shot_map == 0).astype(float)
    return post

def conta_pose(shot_map, sizes, targeting):
    post = np.zeros((10, 10))
    for length in sizes:
        for horizontal in (True, False):
            righe = 10 if horizontal else 10 - length + 1
            colonne = 10 - length + 1 if horizontal else 10
            for i in range(righe):
                for j in range(colonne):
                    cells = [(i, j + k) if horizontal else (i + k, j)
                             for k in range(length)]
                    if any(shot_map[a, b] == 1 for a, b in cells):
                        continue               # la posa copre un colpo a vuoto
                    n_hit = sum(shot_map[a, b] == 2 for a, b in cells)
                    if targeting and n_hit == 0:
                        continue               # non spiega nessun colpo a segno
                    peso = 10 ** n_hit
                    for a, b in cells:
                        if shot_map[a, b] == 0:
                            post[a, b] += peso
    return post

def get_best_coordinates(posterior_map):
    return np.unravel_index(np.argmax(posterior_map), posterior_map.shape)

def bayesian_search_strategy(grid):
    shot_grid = np.zeros((10, 10), dtype=int)
    hits = turns = 0
    while hits < 17:
        i, j = get_best_coordinates(update_posterior(shot_grid, SIZES))
        shot_grid[i, j] = 2 if grid[i, j] else 1
        turns += 1
        if grid[i, j]:
```

```
43                 hits += 1
44         return shot_grid, turns
```

Osservazione (dov'è il teorema di Bayes). Il nome "bayesiana" è giustificato anche se nel codice non compare la formula. Il prior è uniforme su tutte le disposizioni possibili; una cella risultata a vuoto ha verosimiglianza 0 per ogni disposizione che la occupa, quindi tutte quelle disposizioni vengono eliminate dal posteriore. Contare le pose sopravvissute è esattamente calcolare la probabilità a posteriori marginale di ogni cella, a meno della costante di normalizzazione, che non serve perché si cerca solo l'*argmax*.

Attenzione (il posteriore può azzerarsi, e il programma si blocca). Scritta nel modo ovvio (solo il conteggio, senza i due `if` iniziali) questa strategia entra in **ciclo infinito**. Succede così: una nave da 2 celle viene colpita tutta e si trova circondata da celle già esplorate. I suoi due colpi a segno restano "aperti" perché non si tiene traccia degli affondamenti, quindi si è in modalità target; ma l'unica posa che li spiega non tocca alcuna cella inesplorata, e quindi non assegna voti a nessuno. Se tutti i colpi a segno aperti sono in questa condizione il posteriore è *tutto zero*, `argmax` restituisce sempre $(0, 0)$, si spara all'infinito sulla stessa cella e il contatore dei colpi a segno non avanza più. Il rimedio è il ripiego in due passi: se il posteriore in modalità target è nullo si ricalcola in modalità hunt, e se è nullo anche quello si spara su una qualsiasi cella inesplorata. Vale la pena saperlo perché il sintomo (il notebook che non finisce mai) non assomiglia per niente alla causa.

Attenzione (il peso 10^{n_hit} è un'euristica, non Bayes). Il filtro sui colpi a vuoto è un calcolo bayesiano esatto; il peso 10^{n_hit} no, è una scorciatoia per dire "le pose che spiegano più colpi sono molto più plausibili". Il calcolo rigoroso richiederebbe di enumerare le disposizioni *congiunte* delle cinque navi compatibili con tutti i dati, che sono troppe per essere contate a ogni turno. Vale la pena saperlo, perché in sede d'esame la differenza tra "esatto" e "approssimato" è il tipo di cosa su cui si viene interrogati.

Approfondimento (che cosa manca rispetto alle regole vere). Nelle regole, quando una nave affonda l'avversario deve dichiarare *quale* nave era. L'implementazione sopra non usa questa informazione: tiene sempre tutte e cinque le lunghezze in `SIZES`. Tenere traccia delle navi affondate e rimuoverle dalla lista (e marcare come "chiusi" i loro colpi a segno) rende il posteriore più preciso e fa guadagnare qualche turno. È il primo miglioramento da provare.

2.9.5 Risultati misurati

Le tre strategie sono state eseguite sulle stesse partite generate casualmente (2000 partite per le prime due, 300 per la ricerca bayesiana, che è molto più pesante: ricostruisce il posteriore da zero a ogni turno).

	Casuale	Hunt and target	Ricerca bayesiana
Partite simulate	2000	2000	300
Turni medi	95,3	59,8	54,2
Mediana	97	61	54
Minimo	66	30	29
Massimo	100	80	77
Partite chiuse entro 50 turni	0%	17,6%	35,0%

Il salto grosso è il primo: passare dal caso a *hunt and target* fa risparmiare circa 35 turni, cioè oltre un terzo della partita. Il secondo salto è molto più piccolo, circa 6 turni in media, ma cambia la *forma* della distribuzione: la quota di partite chiuse entro 50 turni raddoppia (dal 17,6% al 35%). La ricerca bayesiana non è tanto più veloce nel caso tipico, quanto più **affidabile**: sbaglia meno spesso in grande.

Attenzione (questi 54 turni si possono migliorare parecchio). Le implementazioni pubblicate di ricerca bayesiana per la battaglia navale arrivano attorno ai 42–45 turni. La differenza viene quasi tutta da due semplificazioni fatte sopra: non si tiene traccia delle navi affondate (quindi il posteriore continua a cercare navi che non ci sono più) e il peso 10^{n_hit} è un'euristica. Il numero riportato è quello *effettivamente misurato con questo codice*, non il migliore ottenibile: vale la pena saperlo prima di citarlo.

2.9.6 Che cosa portarsi dietro

- Le tre strategie sono tre livelli di uso dell'informazione: nessuno (casuale), locale (hunt and target), globale (bayesiana).
- La ricerca bayesiana non ha regole scritte a mano: si limita a massimizzare l'utilità attesa rispetto a una distribuzione aggiornata. Il comportamento "intelligente" (sparare a scacchiera all'inizio, seguire l'orientamento della nave dopo due colpi) **emerge** dal calcolo.
- È lo schema che si ritrova nelle vere applicazioni di *search theory*: i paper in `labs/02-battleship/papers/` raccontano come la stessa idea sia stata usata per ritrovare il relitto del volo Air France 447.

Riepilogo della lezione

- Le preferenze sono una relazione $\succsim$ su X; se è completa e transitiva è una *relazione di preferenza*, e da essa si derivano $\succ$ e $\sim$.
- **Teorema 1**: con X finito, $\succsim$ è rappresentabile da $u : X \to \mathbb{R}$ se e solo se è completa e transitiva. L'utilità è *ordinale*: invariante per trasformazioni strettamente crescenti, quindi i numeri non si interpretano.
- Con l'incertezza si passa alle *lotterie*. Una lotteria composta si riduce a una semplice moltiplicando le probabilità lungo i rami.
- **Teorema VNM**: $\succsim$ su $\mathcal{L}$ ammette utilità attesa $U(L) = \sum_i p_i u(x_i)$ se e solo se vale ordine di preferenza, continuità e indipendenza. L'utilità è *cardinale*: unica a meno di trasformazioni affini $aU + b$ con $a > 0$.

- Il *paradosso di Allais* mostra che le persone violano l'indipendenza: VNM è una teoria normativa, non descrittiva.
- Atteggiamento verso il rischio = curvatura di u: lineare $\to$ neutrale, concava $\to$ avverso, convessa $\to$ amante. Con $u(x) = \sqrt{x}$ la condivisione del rischio e l'assicurazione creano valore per entrambe le parti (premio $P \in [50, 75]$ nell'esempio).
- **Lab**: la ricerca bayesiana sceglie a ogni turno la cella di massima utilità attesa, aggiornando il posteriore con i colpi a vuoto e a segno.

Prossima lezione: giochi non cooperativi in forma normale, *best response* ed equilibrio di Nash.